

Actividad 1

Parte 1:

1. Identificación de Pares: Enumera tres claves (keys) presentes en el nivel superior del objeto JSON.
 - "hostname": "R1-Core"
 - "name": "GigabitEthernet0/0"
 - "name": "GigabitEthernet0/1"
2. Tipos de Datos: Identifica un ejemplo de cada uno de los siguientes tipos en el código: String, Number, Boolean y Null.
 - "hostname" es string
 - "uptime_days" es int
 - "managed_by_apic" es boolean
 - "interfaces" es una lista
 - "name": es un string
 - "description" es un string
 - "enabled" es un boolean
 - "mtu" es un int
 - "name" es un string
 - "description": null, "enabled": false, "mtu": 1500

Parte 2: Detección de Errores de Sintaxis

En la automatización de redes, un error de sintaxis en un payload JSON puede causar que la configuración de un dispositivo falle. Analiza el siguiente fragmento de código destinado a configurar una VLAN y responde.

```
{  
  "vlan_id": 10  
  "name": "Usuarios_Ventas"  
  "tagged": true,  
  "description": "VLAN para el departamento de ventas",  
}
```

3. ¿Qué errores de sintaxis impiden que este JSON sea válido según los estándares de red y programación? Selecciona las dos opciones correctas.

A) El valor de 'vlan_id' debería estar entre comillas dobles obligatoriamente.

B) Las claves como 'vlan_id' no deben llevar guiones bajos.

C) Existe una coma final (trailing comma) después del valor de 'description'.

D) Faltan comas para separar los pares clave-valor después de 'vlan_id' y 'name'.

4. Reflexión Técnica: Explica por qué es crítico que un controlador como Cisco DNA Center reciba un JSON perfectamente validado antes de aplicar cambios en la red.

Es crítico que un controlador de red reciba correctamente la información JSON, debido a que en base a este se tomaran decisiones de cambio en la red que pueden perjudicar a los usuarios si es que no se esta correctamente configurado.

Actividad 2:

Parte 1: Análisis de Inventario

Imagina que estás automatizando la auditoría de una red empresarial. Has recibido el siguiente objeto JSON que representa el inventario actual de dispositivos. Tu objetivo es identificar qué equipos están fuera de línea (down) para generar un reporte de incidentes.

```
[  
{"hostname": "Core-SW01", "ip": "10.0.0.1", "status": "up"},  
{"hostname": "Dist-SW02", "ip": "10.0.0.2", "status": "down"},  
{"hostname": "Access-SW03", "ip": "10.0.0.3", "status": "up"},  
{"hostname": "Edge-R01", "ip": "172.16.1.1", "status": "down"}  
]
```

1. Escribe el pseudocódigo o completa el script de Python necesario para procesar la variable inventario_json (que contiene el texto anterior). El script debe filtrar e imprimir únicamente los hostnames de los dispositivos cuyo estado sea 'down':

```
#Imprimir todos los que esten en down
```

```
print("Inventario en status 'down':")
```

```
for d in inventario:
```

```
    if d["status"] == "down":
```

```
        print(d["hostname"])
```

Parte 2: Comparativa de Formatos (JSON vs. XML)

En redes modernas, REST APIs suelen preferir JSON por ser más ligero, pero muchos sistemas heredados o protocolos como NETCONF utilizan XML. Observa cómo se estructuran los mismos datos en ambos formatos.

Formato JSON Formato XML

Formato JSON	Formato XML
<pre>{ "hostname": "R1", "status": "up" }</pre>	<pre>xml <device> <hostname>R1</hostname> <status>up</status> </device></pre>

2. Utiliza el siguiente organizador gráfico para clasificar las características de JSON y XML

basándote en la comparativa anterior y el contenido del Tema 4.

Comparativa JSON vs XML	
JSON	XML
Hace uso de corchetes al declarar nuevas variables con sus atributos, asignándoles valores como en los diccionarios python	Hace uso de etiquetas de inicio y fin para especificar las variables y sus atributos, así como el formato usado en HTML